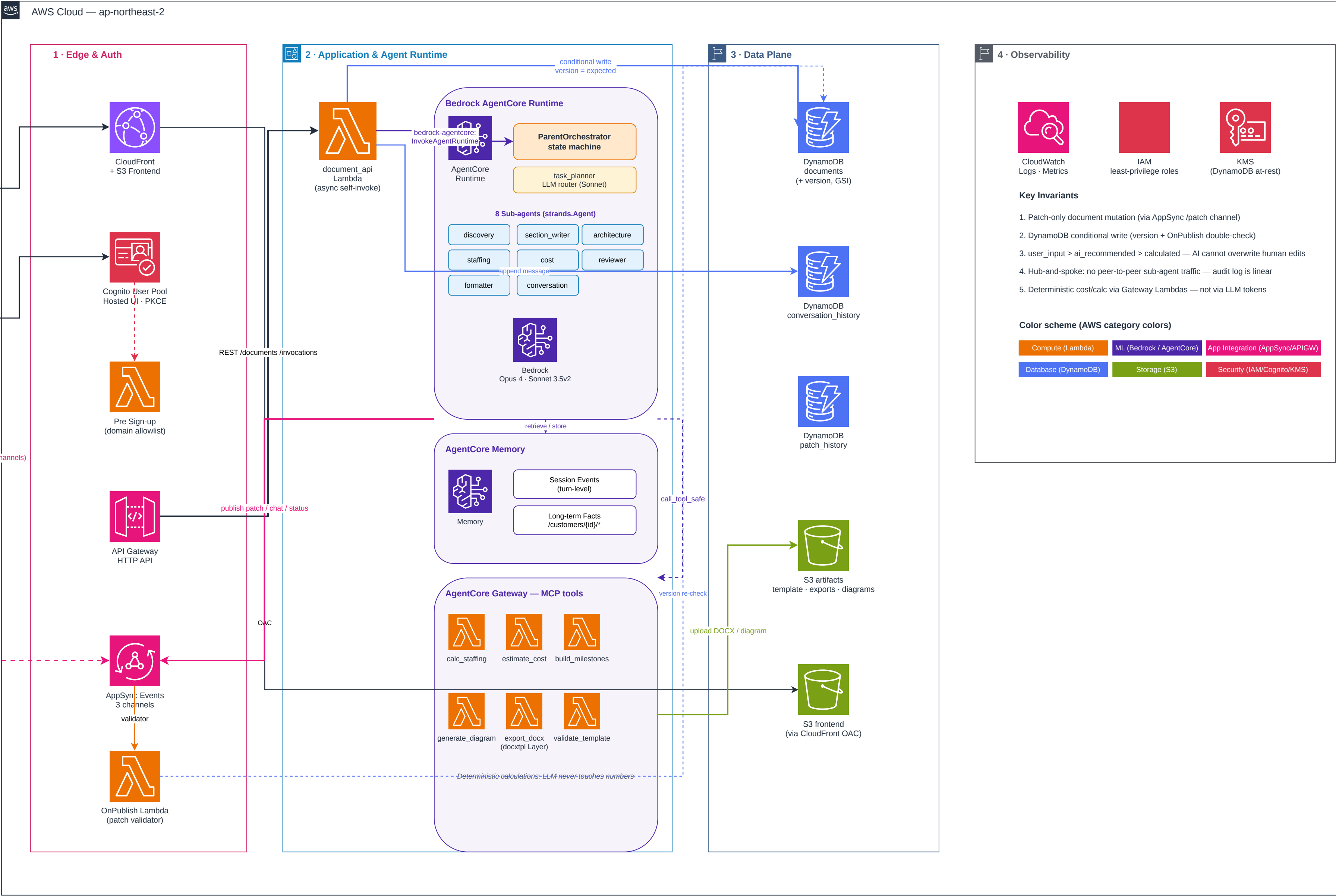


# Spaceflow — APN PoC Project Plan Co-Editing Platform

Region: ap-northeast-2 (Seoul) · Account: 626635430480 · Auth: Cognito (PKCE)



● MEGAZONECLOUD · APN GenAI IC

# Doc Agent 설계 요약

AgentCore 기반 멀티에이전트 문서 생성 시스템

대상 · 발표 평가 / 기술 리뷰

범위 · 아키텍처 / 데이터 모델 / 핵심 흐름 / 운영 / 제약

리전 · ap-northeast-2 · 계정 · 589783680743

문서 일자 · 2026-05

AgentCore Runtime

AgentCore Gateway · MCP

AgentCore Memory

Claude Opus 4.6 / Sonnet 3.5 v2

DynamoDB · AppSync Events

React + Vite + Zustand

# 1. Executive Summary

Doc Agent는 AWS 파트너가 **APN PoC Project Plan**을 구조화된 대화로 작성·검증·내보내기 하도록 돕는 멀티에이전트 시스템이다. 사용자 채팅 한 줄이 **의도 분류 → 서브에이전트 위임 → JSON Patch 생성 → 낙관적 잠금 반영 → 실시간 화면 갱신**으로 이어진다.

7

서브에이전트

7

Gateway MCP 도구

5

DynamoDB 테이블

3

AppSync 채널

5

상태머신 단계

## 1.1 해결하는 문제

- 수작업 문서 작성에서 발생하는 **누락 · 표현 차이 · 계산 오류** (비용, 인건비, 마일스톤)
- 긴 이메일 스레드/회의록/SOW 초안에서 필요한 정보를 **한 번에 구조화**하기 어려움
- AWS APN 심사 기준을 사전에 **체계적으로 리뷰**할 도구 부재

## 1.2 핵심 차별점

**멀티에이전트 오케스트레이션**

Hub-and-spoke 패턴. 사용자 메시지 → Parent(Opus 4.6)가 의도 분류 → 전문 서브에이전트에 위임. 모든 호출은 Parent를 통해 audit log로 추적.

**실시간 공동 편집 UX**

AppSync Events 3채널(patch/status/chat)로 서브에이전트 진행 상황과 문서 변경을 **즉시 반영**. 30초 API Gateway 하드 리밋을 WebSocket 경로로 우회.

**낙관적 잠금 + 버전 관리**

DynamoDB `ConditionExpression` 으로 동시 편집 충돌을 안전하게 감지. 모든 변경은 JSON Patch 단위로 `patch-history` 에 기록.

**AgentCore 기반 운영**

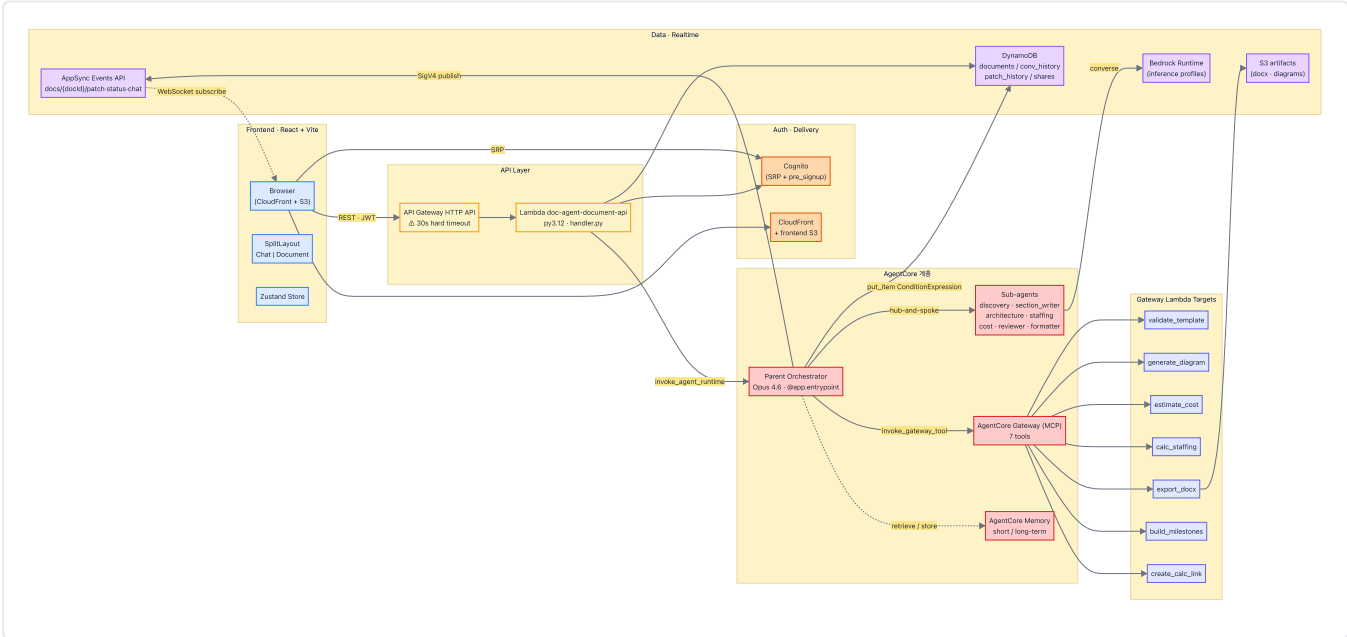
Runtime(직접 코드 배포) + Gateway(MCP 도구 규격) + Memory(단기/장기). 컨테이너 빌드 없음, Lambda를 MCP 도구로 노출.

## 1.3 평가 포인트

| 영역      | 무엇을 보여주는가                                                | 관련 섹션      |
|---------|----------------------------------------------------------|------------|
| 시스템 설계  | 관심사 분리(동기 CRUD vs LLM 오케스트레이션), hub-and-spoke, 상태머신      | \$2, \$3   |
| 데이터 모델  | FieldValue 4-property 패턴, 낙관적 잠금, JSON Patch 기반 mutation | \$4        |
| 운영 · 비용 | degraded mode, inference profile fallback, 리전 제약 회피      | \$6        |
| 확장성     | 에이전트 레지스트리 기반 라우팅, MCP 규격 도구                             | \$3.2, \$5 |

## 2. 시스템 아키텍처

### 2.1 전체 구성



관심사 분리 원칙. Lambda(handler.py)는 동기 CRUD·공유·export 등 결정적 로직을, AgentCore Runtime은 LLM 오케스트레이션을 담당한다. Lambda에서 `from agent.* import`는 금지 — Runtime은 별도 배포 아티팩트.

### 2.2 계층별 역할

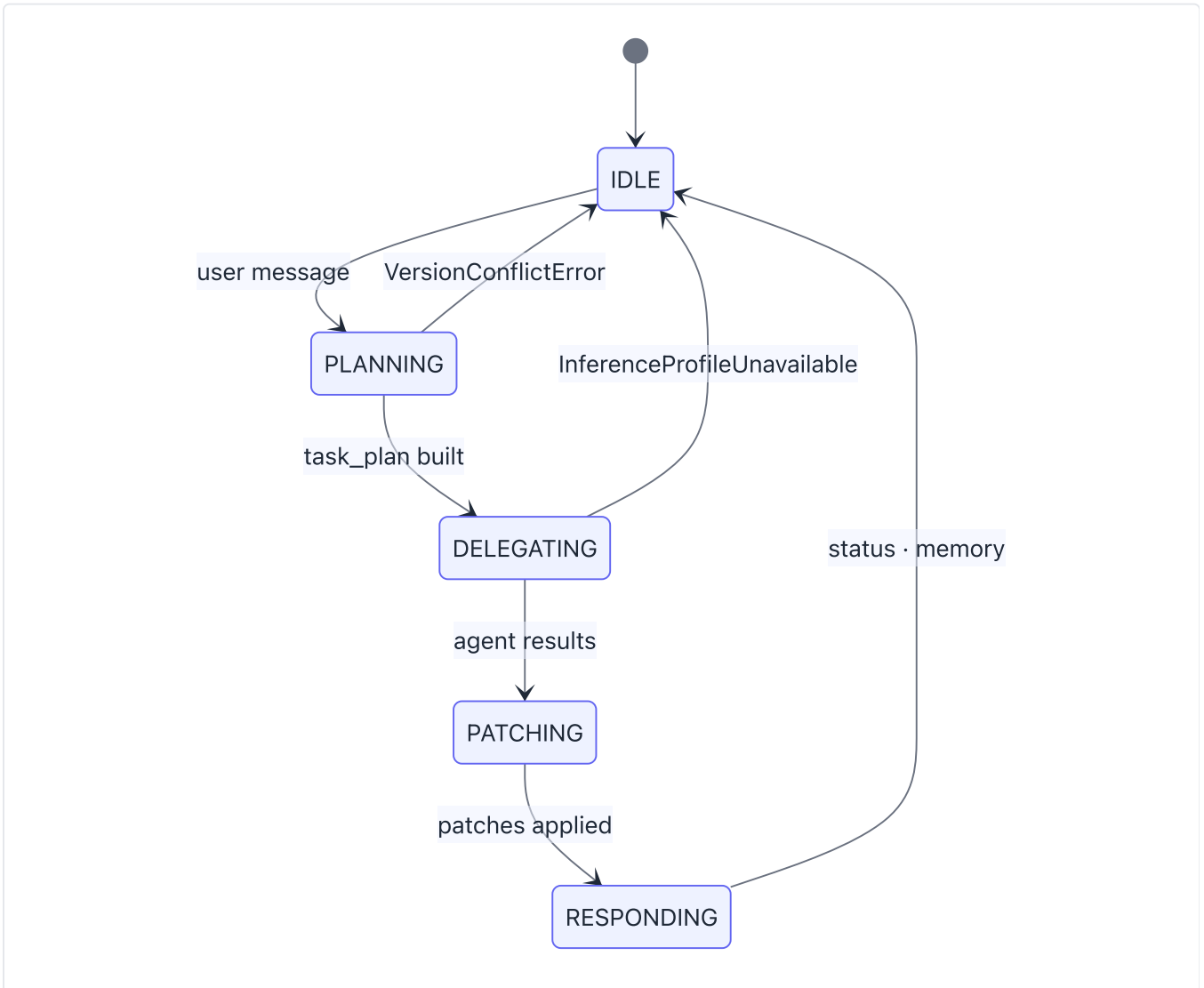
| 계층       | 기술                                 | 책임                                                                   |
|----------|------------------------------------|----------------------------------------------------------------------|
| Frontend | React 19 · Vite · Zustand          | 대화·문서 UI, WebSocket 구독, 낙관적 UI 업데이트                                  |
| Delivery | CloudFront + S3 (frontend-*)       | SPA 정적 호스팅, 403/404 → index.html                                     |
| Auth     | Cognito User Pool                  | SRP 인증, pre_signup Lambda로 도메인 화이트리스트                                |
| API      | API Gateway HTTP API + Lambda      | REST CRUD, 공유, chat proxy, docx export                               |
| Runtime  | AgentCore Runtime + strands-agents | Parent Orchestrator, 서브에이전트, 상태머신                                    |
| Tools    | AgentCore Gateway (MCP)            | Lambda를 MCP 도구로 등록, SigV4 인증                                         |
| Memory   | AgentCore Memory                   | 세션 이벤트(단기), 고객 사실(장기, namespace 기반)                                  |
| Data     | DynamoDB × 5                       | documents(+GSI), conv_history, patch_history, shares, pending_shares |

| 계층       | 기술                           | 책임                                           |
|----------|------------------------------|----------------------------------------------|
| Realtime | AppSync Events API           | patch/status/chat 3채널, API Key(구독) + IAM(발행) |
| Storage  | S3 (artifacts-*, frontend-*) | 에이전트 코드 ZIP, 다이어그램/DOCX, 빌드 자산               |

## 3. 에이전트 설계

### 3.1 Parent Orchestrator 상태머신

모든 사용자 메시지는 `handle_message()` 한 함수에서 다섯 단계를 거친다. 각 단계 사이에서 AppSync로 상태를 발행하여 프론트가 진행을 시각화한다.



### 3.2 서브에이전트 카탈로그

에이전트 정의는 `agent/data/presets/agent_registry.json`에 있다. 라우팅은 하드코딩된 키워드가 아니라 **LLM 기반 task\_planner**가 사용자 메시지 + 레지스트리를 받아 결정한다. 짧은 메시지( <300자 )만 규칙 기반 단축 경로를 탄다.

| 에이전트      | 모델            | 담당 섹션/액션                                                                               |
|-----------|---------------|----------------------------------------------------------------------------------------|
| discovery | Sonnet 3.5 v2 | 이메일/SOW/회의록 → cover, meta, executive_summary, stakeholders, scope, milestones 등 초기 채우기 |

| 에이전트           | 모델                                  | 담당 섹션/액션                                                                                   |
|----------------|-------------------------------------|--------------------------------------------------------------------------------------------|
| section_writer | Sonnet 3.5 v2                       | executive_summary · success_criteria · assumptions · scope · milestones · acceptance 작성/수정 |
| architecture   | Sonnet 3.5 v2                       | .drawio 분석, AWS 서비스 구성 설계, Gateway의 generate_diagram 호출                                    |
| staffing       | Sonnet 3.5 v2                       | 역할/인원/할당률/페이지별 시간, role_pool 프리셋 기반 추천                                                     |
| cost           | Sonnet 3.5 v2                       | 인건비(deterministic) + AWS 비용(Calculator MCP) + fallback card                                |
| reviewer       | Sonnet 3.5 v2 · Nova Lite(fallback) | APN 심사 룰 매트릭스 기반 PASS/WARNING/FAIL 판단                                                      |
| formatter      | Sonnet 3.5 v2                       | APN DOCX 템플릿으로 최종 문서 생성 (docxtpl)                                                          |
| conversation   | -                                   | 인사·도움말·잡담 등 문서와 무관한 대화                                                                     |

**Hub-and-spoke 강제.** 서브에이전트끼리는 절대 직접 호출하지 않는다. `delegate_task()` 가 모든 호출의 단일 진입점이며, 각 호출은 `_audit_log` 에 기록되어 `user_message → task → patches` 매핑이 감사 가능하다.

### 3.3 LLM 라우터(task\_planner) 설계

- 레지스트리 주입 — 시스템 프롬프트에 `agent_registry.json` 전체가 포함되어, JSON만 수정하면 라우팅이 바뀐다 (코드 변경 불필요).
- 화이트리스트 검증 — LLM이 존재하지 않는 에이전트 이름을 반환하면 (예: `review_agent`) 거부하고 fallback.
- 긴 입력 보호 — 이메일 본문처럼 300자가 넘는 메시지는 키워드 매칭을 건너뛰고 LLM에만 의존(스트레이 키워드로 오라우팅 방지).

## 4. 핵심 메시지 흐름

아래 시퀀스는 "이메일 기반으로 초기 정보 채워줘" 같은 한 번의 사용자 메시지가 어떻게 문서 변경과 실시간 UI 갱신으로 이어지는지 보여준다.



**Syntax error in text**  
mermaid version 10.9.1

### 4.1 채널 분리와 30초 제약 우회

API Gateway HTTP API는 30초 integration timeout이 하드 리밋이다. Runtime이 여러 서브에이전트를 돌리면 30초를 쉽게 넘긴다.

- **HTTP 응답**은 간결하게 (채팅 텍스트 버전만)
- 실제 문서 변경은 `docs/{docId}/patch` 로 AppSync 발행
- 서브에이전트 진행 상황은 `docs/{docId}/status` 로 ProgressPublisher가 계속 송출
- 스트리밍 챗은 `docs/{docId}/chat` 에 토큰 델타로 발행

**Runtime 로깅 제약.** AgentCore Runtime 내부 `print()` · `logger` 는 CloudWatch에 잘 올라오지 않는다(알려진 플랫폼 제한). 확실한 디버깅이 필요한 메시지는 `handler.py`(Lambda 환경)에서 찍고, Runtime은 블랙박스로 취급한다.

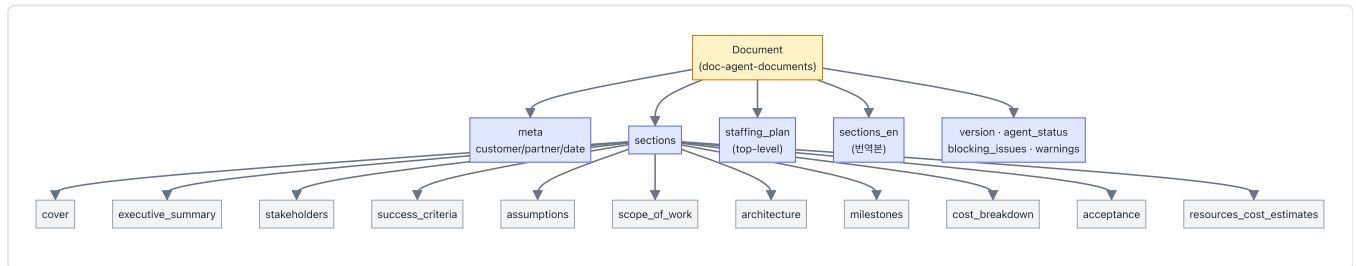
### 4.2 오류 경로

- **VersionConflictError** — 다른 세션이 먼저 버전을 올린 경우. 채팅으로 "새로고침 후 다시 시도" 안내.
- **InferenceProfileUnavailableError** — 지정한 Bedrock inference profile 일시 사용 불가. fallback profile로 재시도, 두 개 모두 실패 시 degraded 응답.
- **Gateway tool failure** — `GatewayToolError` 는 문서 상태를 부분 변경하지 않도록 보장. 비용 톨 실패 시 **FallbackCard**(프리셋 기반 추정)로 대체.
- **Memory failure** — degraded mode로 계속 동작. 기본 기능(CRUD·라우팅)은 영향 없음.



## 5. 데이터 모델

### 5.1 Document\_State 구조



### 5.2 FieldValue 4-property 패턴

사용자 편집 가능한 모든 필드가 공유하는 구조. AI 추천과 사용자 입력을 동시에 보존하고, 어떤 값이 화면에 쓰일지는 우선 순위로 해소된다.

```
{
  "user_input":      "사용자가 직접 쓴 값",      // 최우선
  "ai_recommended": "에이전트가 제안한 값",
  "calculated":      "자동 계산 결과",          // 비용·합계
  "status": "empty | recommended | user_modified | confirmed | calculated",
  "user_edited":     false,
  "reason":          "변경 사유",
  "source_patterns": ["preset_name"],
  "confidence":      0.85
}
```

화면에는 `user_input` || `ai_recommended` || `calculated` 순으로 해소된 값이 표시된다  
( `_resolve_field_value` ). 사용자가 편집하면 `user_input` 만 바뀌고 `ai_recommended` 는 보존되어 비교 가능하다.

### 5.3 낙관적 잠금 (Optimistic Locking)

모든 쓰기는 `version` 을 포함하는 조건부 PUT이다. 두 세션이 같은 버전을 기준으로 동시에 쓰면 하나만 성공한다.

```
self._table.put_item(
    Item=item_with_version_n_plus_1,
    ConditionExpression=Attr("version").eq(expected_version),
)
# ConditionalCheckFailedException → VersionConflictError
```

### 5.4 부수 테이블

| 테이블                                         | 키                                   | 용도                                                                          |
|---------------------------------------------|-------------------------------------|-----------------------------------------------------------------------------|
| <code>doc-agent-conversation-history</code> | PK document_id · SK session_id      | bounded_window N턴 + messages 배열. 프론트는 최근 N턴만 전송, 나머지는 AgentCore Memory가 보충. |
| <code>doc-agent-patch-history</code>        | PK document_id · SK patch_id        | version_before / version_after · operations 배열. 감사·롤백 기반.                   |
| <code>doc-agent-document-shares</code>      | PK document_id · SK grantee_user_id | viewer/editor 역할, <code>granted_by</code> / <code>granted_at</code> .       |
| <code>doc-agent-pending-shares</code>       | PK document_id · SK grantee_email   | 미가입자 초대 대기. <code>post_confirmation</code> Lambda가 승격.                      |

**상태 원칙.** DynamoDB = source of truth. AppSync는 현재 보는 문서의 실시간 보너스 채널이다. 문서 전환 시에는 DynamoDB에서 최신을 받고, 이후 WebSocket으로 증분을 받는다.

# 6. Gateway · Memory · 외부 계층

## 6.1 AgentCore Gateway — MCP 도구 레이어

Parent·Architecture·Cost·Formatter 에이전트가 공유하는 도구 호출 계층. Gateway에 도구 스키마와 **Lambda target**을 등록하면, Runtime은 `invoke_gateway_tool(gatewayId, toolName, input)` 한 API로 MCP 규격의 도구를 호출한다.

| MCP 도구                                     | Lambda                              | 입력 요지                          | 출력 요지                      |
|--------------------------------------------|-------------------------------------|--------------------------------|----------------------------|
| <code>validate_template_constraints</code> | <code>validate_template</code>      | <code>document_snapshot</code> | blocking/warning issues    |
| <code>generate_architecture_diagram</code> | <code>generate_diagram</code>       | services, connections          | S3 키 · PNG URL             |
| <code>estimate_cost</code>                 | <code>estimate_cost</code>          | services, usage                | MRR/ARR breakdown          |
| <code>calculate_staffing_cost</code>       | <code>calc_staffing</code>          | roles, rates, phases           | 총 시간/비용 + 역할 별             |
| <code>export_docx</code>                   | <code>export_docx</code>            | <code>document_state</code>    | APN 템플릿 DOCX URL           |
| <code>build_milestone_summary</code>       | <code>build_milestones</code>       | scope, staffing                | phases · deliverables      |
| <code>create_calculator_link</code>        | <code>create_calc_link(Node)</code> | services · usage               | AWS Pricing Calculator URL |

안전성 보증. `AgentCoreGatewayClient.call_tool_safe()` 는 실패 시 `(None, error_msg)` 를 돌려주며 문서 상태를 부분 변경하지 않는다. 호출자가 FallbackCard 같은 대체 경로를 선택할 수 있다.

## 6.2 AgentCore Memory

### Short-term (세션)

`create_event` 로 `{role, content}` 를 `session_id` 스코프에 저장. 세션별 자동 관리, 다음 턴 프롬프트에 보충.

### Long-term (고객 사실)

`batch_create_memory_records` 로 `/customers/{name}/` namespace에 저장. 동일 고객의 다음 문서 작성 시 `retrieve_memory_records` 로 자동 불러옴.

모든 Memory 호출은 `_safe_call` 래퍼로 감싸져 있다. 실패 시 `on_degraded` 콜백이 발동되고 시스템은 bounded session history만으로 계속 동작(degraded mode).

## 6.3 Bedrock 모델 전략 (서울 리전)

서울 리전 제약. 직접 모델 ID로 on-demand 호출은 `ValidationException` 을 던진다. 반드시 **inference profile** 을 사용해야 한다 ( `apac.*` 또는 `global.*` prefix).

| 역할                  | Primary                                                     | Fallback                                              |
|---------------------|-------------------------------------------------------------|-------------------------------------------------------|
| Parent Orchestrator | <code>global.anthropic.claude-opus-4-6-v1</code>            | 환경변수로 지정 가능                                           |
| Child / Router      | <code>apac.anthropic.claude-3-5-sonnet-20241022-v2:0</code> | 환경변수로 지정 가능                                           |
| Reviewer            | Child 모델과 동일                                                | <code>apac.amazon.nova-lite-v1:0</code> — 비용 절감 + 가용성 |

## 6.4 프론트엔드 실시간 반영

AppSync Events API는 WebSocket 기반 이벤트 채널이다. 프론트는 subprotocol 방식 핸드셰이크( `header-{base64url json}` )로 연결한다.

- `docs/{docId}/patch` → `applyPatches()` 로 Zustand 상태에 머지
- `docs/{docId}/status` → 에이전트 진행 UI 업데이트 (processing/idle/degraded/error)
- `docs/{docId}/chat` → 스트리밍 챗 렌더링

**이벤트 페이로드 형식 주의.** `data.event` 는 배열이 아니라 **단일 JSON 문자열**이다. 배열처럼 순회하면 글자 단위로 파싱되는 버그가 발생. 프론트에서 `Array.isArray(raw) ? raw : typeof raw === 'string' ? [raw] : []` 패턴 사용.

## 7. 주요 설계 결정과 근거

| 결정                                                                            | 대안                                        | 선택 이유                                                                                           |
|-------------------------------------------------------------------------------|-------------------------------------------|-------------------------------------------------------------------------------------------------|
| Lambda와 Runtime을 분리 (handler.py가 <code>agent/*</code> import 금지)              | Lambda 하나에 모두 패키징                         | Lambda cold start, 배포 크기, 동기 CRUD와 장시간 LLM 작업의 관심사 분리. Runtime은 컨테이너 없이 직접 코드 배포.               |
| 서브에이전트 <b>logical</b> 구현 (Parent Runtime 내 함수)                                | 각 서브에이전트 독립 Runtime                       | 초기 비용·운영 복잡도 최소화. 추후 병목 발생 에이전트만 독립 Runtime 분리 가능.                                              |
| <b>AppSync Events API</b> (v2, CloudFormation 경유)                             | AppSync GraphQL subscription / MQTT / SSE | 채널 기반 pub/sub이 문서별 격리에 자연스러움. Terraform aws provider ~5.0은 Events 미지원이라 CloudFormation 스택으로 우회. |
| <b>JSON Patch</b> 단위 mutation                                                 | 전체 문서 PUT                                 | 부분 변경 감사, 동시 편집 merge 용이, AppSync 페이로드 최소화.                                                     |
| 낙관적 잠금 (version 필드 + ConditionExpression)                                     | Pessimistic locking, DAX lease            | 다중 세션 동시 편집 빈도가 낮고, 충돌 시 사용자 재시도가 허용되는 워크플로. DynamoDB native 기능 활용으로 운영 비용 0.                   |
| <b>LLM 기반 라우터</b> + agent_registry.json                                       | 하드코딩 키워드 매칭                               | 긴 이메일 본문에서 스트레이 키워드로 오라우팅되는 문제 해결. 라우팅 규칙을 JSON 한 파일로 관리.                                       |
| FieldValue <b>4-property</b> 패턴 (user_input/ai_recommended/calculated/status) | 단일 값 필드                                   | AI 추천과 사용자 입력 공존, 편집 이력 추적, UI에서 "AI 추천" 배지·되돌리기 지원.                                            |
| <b>Inference profile</b> 사용 + fallback 체인                                     | 직접 모델 ID                                  | ap-northeast-2에서 direct model ID on-demand 불가. primary → fallback 이중화로 지역 장애 대응.                |
| <b>Reviewer</b> 는 fire-and-forget + DynamoDB 결과 작성                            | Lambda가 Runtime 응답까지 대기                   | 룰 매트릭스(수십 개) × LLM 판단이 30초를 초과. 결과는 DynamoDB review_job 아이템에 저장되고, 프론트가 폴링/구독으로 읽음.             |

### 7.1 트레이드오프

- **AgentCore Runtime** 로깅 제약 — 내부 `print` 가 CloudWatch에 올라오지 않음. 디버깅 UX 저하를 감수하되, 핵심 로그는 handler.py(Lambda)에서 남김.

- **API Gateway 30초 하드 리밋** — 늘릴 수 없음. 긴 작업은 모두 AppSync 경로로 전환. Lambda Function URL은 타임아웃이 없지만 API GW의 인증/CORS 통합을 포기해야 하므로 채택하지 않음.
- **Lambda boto3 버전 차이** — Lambda 런타임의 boto3가 로컬보다 오래되어 Memory API 일부 메서드 미지원. 실패 시 degraded mode로 계속 동작.

## 8. 운영 · 배포 · 리스크

### 8.1 배포 순서

1. `deploy-tf.sh` — Terraform으로 DynamoDB · Lambda · API Gateway · Cognito · AppSync(CFN) · S3 · CloudFront 생성
2. `deploy-lambda.sh` — document\_api + 7개 gateway tools Lambda 코드 업로드
3. `deploy-agent.sh` — AgentCore Runtime(코드 zip) + Gateway(target 등록) + Memory CDK
4. `deploy-front.sh` — Vite 빌드 → S3 업로드 → CloudFront invalidation. `VITE_*` env는 Terraform output에서 자동 주입.

계정 격리. 모든 `deploy-*.sh` 는 `_aws_env.sh` 가 `EXPECTED_ACCOUNT=589783680743` 과 실제 계정을 비교해서 다르면 abort. 구 계정(mzadmin, 626635430480)으로의 배포를 사전 차단.

### 8.2 관찰 가능성

- **Lambda CloudWatch Logs:** `[doc-api:action] level=... key=val` 포맷의 구조화 로깅. 시크릿은 남기지 않음(secret-safe).
- **Runtime ProgressPublisher:** 각 서브에이전트 시작/완료, 토큰 델타, 도구 호출을 AppSync로 실시간 송출. UI에 진행 표시.
- **Audit log:** `TaskPlan.execution_log` + `patch-history` 로 사용자 메시지 → 위임 → 패치의 전체 궤적 복원 가능.
- **Standard envelope:** 신규 API는 `{status, message, warnings, missing_inputs, error_reason}` 봉투를 응답에 포함해 프론트 에러 UX 일관화.

### 8.3 보안

- Cognito SRP 인증, JWT를 `Authorization` 헤더로 전달. Lambda가 `X-User-Id` 와 교차 검증.
- 가입 도메인 제한 — `pre_signup` Lambda가 `@mz.co.kr` , `@megazone.com` 만 허용.
- 문서 공유는 `document_shares` / `pending_shares` 테이블로 역할(viewer/editor) 기반 제어.
- AppSync 구독은 API Key, 발행은 IAM SigV4 — 브라우저는 발행할 수 없음.

### 8.4 리스크 & 대응

| 리스크                               | 영향        | 대응                                                                   |
|-----------------------------------|-----------|----------------------------------------------------------------------|
| <b>기술</b> AgentCore Runtime 로깅 누락 | 디버깅 난이도 ↑ | handler.py에서 보강 로깅. Runtime을 블랙박스로 취급. ProgressPublisher로 외부 가시성 확보. |
| <b>기술</b> Bedrock 리전 장애/모델 불가     | 응답 실패     | inference profile fallback 체인. Reviewer는 Nova Lite로 자동 전환.           |

| 리스크                | 영향         | 대응                                                                |
|--------------------|------------|-------------------------------------------------------------------|
| <b>운영</b> 구 계정 혼선  | 잘못된 리소스 수정 | 모든 배포 스크립트 계정 검증. 이전 상태 파일은 <code>.old-account</code> 접미로 격리.     |
| <b>성능</b> 30초 타임아웃 | 사용자 체감 실패  | AppSync 비동기 채널로 작업 분리. 긴 Reviewer는 fire-and-forget + DynamoDB 저장. |
| <b>UX</b> 동시 편집 충돌 | 버전 충돌 에러   | 낙관적 잠금 + 친절한 재시도 메시지. 패치 이력으로 수동 머지 가능.                           |



## 9. 요약 & 다음 단계

### 9.1 한 줄 요약

**Doc Agent = 채팅으로 APN 문서를 쓰는 멀티에이전트 시스템.**

AgentCore Runtime이 Opus 4.6 Parent + 7 Sonnet 서브에이전트를 hub-and-spoke로 조율하고, 결과는 JSON Patch로 DynamoDB에 낙관적 잠금 쓰기 후 AppSync로 실시간 전파된다.

### 9.2 기여 포인트 (평가 관점)

#### 아키텍처 품질

- 관심사 분리: Lambda(동기) ↔ Runtime(LLM)
- Hub-and-spoke로 audit 가능성 보장
- 3채널 AppSync로 30초 한계 우회

#### 데이터 설계

- FieldValue 4-property로 AI/사용자 값 공존
- JSON Patch + version으로 부분 변경 감사
- 낙관적 잠금으로 동시 편집 안전

#### 운영 안전성

- degraded mode로 Memory/Gateway 실패 흡수
- inference profile fallback 체인
- 계정 격리 배포 게이트

#### 확장성

- agent\_registry.json으로 라우팅 수정
- MCP 규격 도구로 Lambda 추가 용이
- 서브에이전트 독립 Runtime 승격 가능

### 9.3 다음 단계 후보

- 병목 서브에이전트(Reviewer, Discovery)를 독립 Runtime으로 분리해 동시성 향상
- Gateway 도구의 HTTP Semantic Cache(결정적 출력 캐싱)로 반복 비용 절감
- AgentCore Observability(trace) 도입 — 현재 ProgressPublisher 커스텀 경로를 공식 trace로 대체
- 공동 편집 CRDT 도입 검토 — 낙관적 잠금 + 수동 재시도의 UX 상한선 극복
- 템플릿 다양화 — APN PoC 외 SOW/IC Approval/SFDC 연동 문서 타입 지원

### 9.4 레퍼런스 문서 (리포지토리)

| 문서                                                            | 용도               |
|---------------------------------------------------------------|------------------|
| <a href="#">.kiro/specs/agentcore-doc-agent-v2/design.md</a>  | v2 전체 설계 원본 (상세) |
| <a href="#">.kiro/specs/frontend-v2-direct-edit/design.md</a> | 프론트 직접 편집 UX 설계  |
| <a href="#">.kiro/specs/document-presets/design.md</a>        | 역할/요율/페이지 프리셋 설계 |

| 문서                                                      | 용도                                  |
|---------------------------------------------------------|-------------------------------------|
| <code>.kiro/steering/*.md</code>                        | 작업/인프라/디버깅/배포/스키마/모델 규칙             |
| <code>docs/design-presentation/diagrams/*.drawio</code> | 이 문서의 아키텍처 다이어그램 소스 (draw.io 편집 가능) |